



Politecnico
di Torino

Algorithms and Programming

Iniziato martedì, 21 dicembre 2021, 17:14

Stato Completato

Terminato martedì, 21 dicembre 2021, 17:14

Tempo impiegato 11 secondi

Valutazione 0,00 su un massimo di 20,00 (0%)

Domanda 1

Risposta non data

Punteggio max.:
2,00

The following recursive function is called by the main program with $n=3$, i.e., as $f(3)$.

```
int f (int n) {  
    if (n<=0) {  
        return (n);  
    }  
    return (f(n-1)+f(n-2)+f(n-3));  
}
```

Following the recursion tree, indicate the value returned by the function to the main program. Please, report your response as a unique integer value (with a sign, i.e., positive or negative). No other symbols must be included in the response. This is an example of the response format: -2

Risposta: ✖

La risposta corretta è : -7

Domanda 2

Risposta non data

Punteggio max.:
4,00

A list of lists represents the relationship between producers and products. Each producer's name is stored into an element of the main list and it has a related secondary list of products. Each product is characterized by an identifier (a string) and its price (a real value), and it is associated to an element of one secondary list.

Write the function to insert a new triple producer-product-price in this list of lists. Notice that the insertion procedure must discriminate among 3 cases:

1. The producer is not present in the database. In that case, the function must introduce the new producer with the new product and its price.
2. The producer is present but it does not have that product. In that case, the function must introduce the new product with its price in the secondary list associated with the right producer.
3. The producer and the product are already present. In this case, the function must update the price of that product for that producer.

The candidate can decide where to insert a new element (on top of the list, on the tail, or in a sorted position) for both the main and the secondary lists. The function must allocate all dynamically defined data fields.

The node data structure and the function prototypes are the following.

The function receives the reference to the pointer to the list of lists (head), and the triple producer name (name), product identifier (id), and price (price) to insert into the list of lists. Please, report the entire C code. Briefly explain (in plain English language, within a C comment) the logic of the code.

```
typedef struct producer_s producer_t;
typedef struct product_s product_t;
struct producer_s {
    char *name;
    producer_t *next;
    product_t *right;
};
struct product_s {
    char *id;
    float price;
    product_t *next;
};
```

```
void insert (producer_t **head, char *name, char *id, float price);
```

A large, empty rectangular box with a thin black border, intended for writing code or providing a solution. It occupies the upper half of the page.

Domanda 3

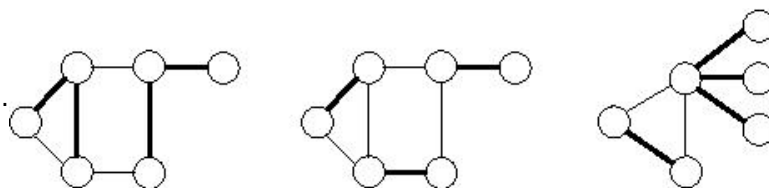
Risposta non data

Punteggio max.:

14,00

In graph theory, an **edge cover** of a graph is a set of edges such that every vertex of the graph is incident to at least one edge of the set. In computer science, the **minimum edge cover** problem is the problem of finding an edge cover of minimum size.

For example, in the next figure, an edge cover is highlighted with bold edges for all three graphs. Nevertheless, in the leftmost graph the edge cover is not minimum, whereas the edge cover is minimum for the other two graphs (middle and right-hand side).



Suppose that an undirected and unweighted graph, with n vertices, is already stored in a proper data structure.

Write a recursive function, named **edge_cover**, which receives the ADT storing the graph (with its number of vertices) and (eventually calling other required functions) displays a minimum cover for such a graph.

Moreover, indicate which is the ADT used to store the graph to be as efficient as possible to run function **edge_cover**. Report its definition in C language and clearly specify which type of ADT and which type of **data hiding** is adopted. Describe the **client-implementation** structure (what it is defined or declared in the C file and what in the header files).

Furthermore, indicate the format used to report the final edge cover found by function **edge_cover**.

Please, report the code used to represent the ADT for the graph and the entire code for the procedure `edge_cover` (and all functions called by it). Explain (in plain English language) the type of ADT used, and the logic of the code (e.g., the combinatorics principle that has been used).

A large, empty rectangular box with a thin black border, intended for the user to write their programming code. It occupies the upper half of the page.